Árbol tipo C4.5 y bosque aleatorio sobre Heart Disease

Celeste Martin Rodich[1], Matías Emmanuel Machado Benítez[1], Camila Aylen Zeniquel Martinelli[1], Nadine Andrea Peralta Ruiz[1], and Mauricio Nicolás Castro[1]

Universidad Tecnológica Nacional, Facultad Regional Resistencia, Argentina

Resumen Se compara un árbol de decisión escrito por el equipo con un bosque aleatorio de scikit-learn sobre Heart Disease [3]: las cuatro sedes juntas, 920 filas, clase binaria ausencia/presencia y 1759 faltantes sin imputar en el cuadro. La métrica que decide es el F1 macro. En 25 particiones estratificadas el bosque llega a $0{,}8157 \pm 0{,}0308$ y el árbol a $0{,}7809 \pm 0{,}0271$. La poda pesimista, con factor de confianza 0,25, baja el árbol de 452 a 71,2 nodos y la brecha de accuracy de 0,1435 a 0,0922; la accuracy media de prueba baja de 0,7850 a 0,7741. Los barridos se hicieron solo sobre el entrenamiento. El árbol no es un C4.5 completo: un atributo se corta una sola vez, la poda solo reemplaza un subárbol por una hoja y el factor t es propio del equipo.

Keywords: árbol de decisión · C4.5 · random forest · gain ratio · poda pesimista

1. Introducción

El trabajo pide comparar dos clasificadores sobre el mismo protocolo. El árbol de decisión lo escribe el equipo. Para el bosque, el enunciado habilita un modelo de biblioteca: "para el algoritmo RF se podrá utilizar un modelo de la librafía Scikit-learn". Se conserva la grafía *librafía* porque así figura en la consigna.

La pregunta es concreta: con los mismos datos y la misma semilla, cuál modelo clasifica mejor, de qué tamaño queda y cuánto tarda en ajustarse. Los datos son las cuatro sedes de Heart Disease [3]. Los investigadores que reunieron cada sede son Andras Janosi (Instituto Húngaro de Cardiología, Budapest), William Steinbrunn (Hospital Universitario de Zúrich), Matthias Pfisterer (Hospital Universitario de Basilea) y Robert Detrano (Centro Médico de Long Beach y Cleveland Clinic). El donante del archivo es David W. Aha (julio de 1988). Detrano y sus coautores publicaron, sobre sedes separadas, una discriminación logística de alrededor del 77 % [2]. Ese número es de ellos. Los de este informe salen de la corrida del 11 de noviembre de 2026.

La numeración de figuras y tablas es la del notebook. Entran la Tabla 2, la Tabla 5 y las Figuras 4, 6, 7, 8, 9 y 10.

2. Algoritmos

2.1. Árbol

El árbol elige el corte con gain ratio: la ganancia de información dividida por la información de la partición, para no premiar a un atributo solo porque abre muchas ramas [4, p. 67]. La entropía se calcula sobre las etiquetas de la clase.

Un atributo categórico abre una rama por valor. Un continuo se parte en dos, con rama izquierda $\leq$ umbral y rama derecha $>$ umbral. Los candidatos son los puntos medios entre valores ordenados donde cambia la clase. En esta corrida esos umbrales coincidieron con los de revisar todos los puntos medios: 374 candidatos en lugar de 454. Después de usar un atributo, el árbol lo saca de la lista, también si es continuo. En cada rama hay un solo corte por columna.

Si falta el valor del atributo que parte, el caso se reparte entre las ramas conocidas en proporción a la masa de cada rama. Cada nodo guarda $t = n_{\text{conocidos}}/n_{\text{total}}$. En una hoja, $t = 1$. Ese factor lo agrega el equipo para la poda con masa fraccionaria. No forma parte de la rutina AddErrs.

La poda es pesimista y corre después de crecer el árbol. En cada nodo se compara el error estimado de dejar el subárbol con el de reemplazarlo por una hoja. Si la hoja no es peor, el nodo se convierte en hoja. El error estimado es $e + u(e, N, CF)$, con $CF = 0{,}25$. La función u sigue AddErrs [6, pp. 278–279]: interpola con CF y, para 0,25, obtiene $z = 0{,}6925$. El código no usa la convención $CF/2$.

2.2. Bosque

El bosque es el de Breiman [1]. Se toma de scikit-learn [5], en la clase `RandomForestClassifier`, y queda envuelto en `RandomForestWrap`. La corrida publicada usa 100 árboles y la semilla 42. Cada árbol ve la raíz cuadrada de los atributos, con remuestreo y hoja mínima de 1. No hay tope de profundidad.

El wrapper imputa las numéricas con la mediana y las categóricas con la moda, y aplica one-hot a estas últimas. Ese paso ocurre dentro de `fit` y de `predict`. El cuadro que ve el árbol no se imputa. Quien compara los modelos les pasa las mismas columnas.

3. Solución y problemas

Los dos modelos exponen `fit(X, y)`, `predict`, `predict_proba` y `score`. `X` es un `DataFrame`. Estos son los problemas que hubo que cerrar para que esa interfaz y las pruebas midieran el árbol que dice el código.

La primera versión del bosque entrenaba con `fit(df, nombre)`, la firma del árbol mínimo, que recibe el cuadro y el nombre de la columna clase. El árbol que entra a las tablas separa atributos y clase. Con las dos firmas, una sola función de evaluación no podía llamar a los dos. El bosque pasó a `fit(X, y)`.

La entropía lee etiquetas, no conteos. Un test le pasaba `[1, 1]` y esperaba 1 bit. Esas dos etiquetas son la misma clase, y la entropía de una clase sola es

0. Para 1 bit hacen falta dos etiquetas distintas, `[0, 1]`. Se dejó la función y se corrigió el test. `gain` le pasa la columna de clase: si la entropía hubiera pasado a leer conteos, un target 0/1 se habría interpretado al revés.

La prueba del núcleo llamaba a `DecisionTreeC45` en una celda anterior a la que define la clase. En el kernel el nombre todavía no existía. La prueba quedó inmediatamente debajo de esa definición.

`predict` devolvía las clases en un arreglo de tipo objeto. Con las etiquetas 0, 1 y 2 de Iris, scikit-learn las toma como destino desconocido y corta el cálculo de la accuracy, aunque el ajuste hubiera terminado bien. Si ese arreglo es de tipo objeto, se rearma: los números quedan como números y el texto sigue siendo texto.

El ROC macro con varias clases usa `multi_class=.ºvr"` sobre la matriz de probabilidades. Con dos clases scikit-learn rechaza esa firma cuando la matriz tiene dos columnas. El valor publicado es el promedio de una curva por clase, que es lo que dibuja la Figura 10.

Un control de la poda exigía que, en el holdout de la semilla 42, la accuracy de prueba no bajara. La poda elige mirando el entrenamiento. El control ahora admite una baja de hasta 3 puntos y sigue exigiendo que el árbol quede con menos nodos. En esta corrida las dos lecturas conviven: la media de las 25 particiones baja de 0,7850 a 0,7741 y ese holdout sube de 0,7880 a 0,7989.

Otro control pedía que un solo corte continuo superara 0,85 de accuracy. Ese piso lo cumplía un atributo de Breast Cancer. En Heart Disease el mejor corte, `chol` $\leq$ 42,50, da 0,5978. El control informa el número.

Pandas 3 guarda el texto como `StringDtype`. Ese tipo no entra en la pregunta «¿es `object` o `category`?», y el árbol categórico mínimo no encontraba columnas. Además, un nodo interno no guardaba la clase mayoritaria: un valor que no estaba en el entrenamiento devolvía vacío y se mezclaba con las etiquetas de texto. El texto cuenta ahora como categoría, y todo nodo guarda la clase mayoritaria. Ahí cae una rama no vista.

Un Run all no puede quedar esperando el permiso de Drive. Esa celda avisa y sigue: los archivos se leen desde el disco.

El árbol que se mide no es un C4.5 completo. Tres diferencias están en el código:

- Un atributo, también si es continuo, se usa una sola vez. C4.5 puede volver a cortar un continuo más abajo, con otro umbral.
- La poda reemplaza un subárbol por una hoja. No hace el otro movimiento de Quinlan, que cambia el subárbol por su rama más usada.
- El factor t es una decisión del equipo para ponderar la masa conocida. No hay que atribuirlo a AddErrs.

El criterio por defecto no impone un gain ratio mínimo y `min_samples_leaf` vale 1. La poda viene apagada. La Tabla 5 la enciende a propósito, con $CF = 0{,}25$. Los códigos de sexo, tipo de dolor, electrocardiograma, angina, pendiente, número de vasos y `thal` se tratan como categorías, no como magnitudes.

4. Metodología

La celda de carga junta los cuatro archivos procesados: Cleveland (303 filas), Hungría (294), Suiza (123) y Long Beach (200). El signo de pregunta queda como faltante. La clase 0 pasa a ausencia (411 filas, 44,7 %) y las clases 1 a 4 pasan a presencia (509, 55,3 %). La línea base de «siempre presencia» es 55,3 %. Hay 1759 faltantes. `ca` falta en el 66,4 % de las filas, `thal` en el 52,8 % y `slope` en el 33,6 %. El colesterol 0 aparece 172 veces; el archivo de nombres no lo define como faltante, así que se deja. Lo mismo vale para el mínimo 0 de la presión arterial, que no se recodificó.

La semilla global es 42. El validador compartido repite 5 veces un pliegue estratificado de 5: son 25 scores. `evaluar_cv` usa ese objeto y rechaza otro splitter. La métrica de comparación es el F1 macro. Con dos clases, la positiva para la matriz y para el ROC es la segunda de `classes_`. En este archivo es presencia.

El corte final es un holdout estratificado 80/20, una sola vez, con la misma semilla: 736 filas de entrenamiento y 184 de prueba. Los barridos ven solo esas 736. Cada grilla mueve un parámetro y deja el resto en el valor por defecto. El elegido es el modelo más chico cuya accuracy media de prueba queda a menos de un desvío estándar del máximo de esa grilla. Si la accuracy sigue subiendo hasta el valor más complejo, la grilla no tiene un óptimo asentado: el informe lo dice y igual aplica la regla. Los tres parámetros del árbol, y los dos del bosque, se eligieron de a uno. El holdout de esa combinación se mide una vez y no se usa para volver a elegir.

La Tabla 2, la Tabla 5 y la Figura 7 se calcularon sobre las 920 filas. El holdout de la Sección 5 es la evaluación final de los valores por defecto. El cruce validado corre en 4 procesos: con 8, la máquina de 8 GB se quedaba sin memoria y la corrida se frenaba. El splitter no cambia.

5. Resultados

En F1 macro el bosque queda arriba: $0{,}8157 \pm 0{,}0308$ contra $0{,}7809 \pm 0{,}0271$ del árbol. La accuracy de prueba es $0{,}8187 \pm 0{,}0305$ y $0{,}7850 \pm 0{,}0265$. El ROC-AUC es $0{,}8839 \pm 0{,}0254$ y $0{,}8195 \pm 0{,}0295$. El árbol tiene 452 nodos y altura 13 en las 25 particiones, con desvío de altura 0: hay 13 atributos y cada uno sale de la lista después del corte, así que el camino más largo usa una vez cada columna. El bosque, sumando sus 100 árboles, tiene unos 28 876 nodos. Ajustar el árbol lleva $50{,}59 \pm 9{,}80$ segundos por pliegue y el bosque $0{,}484 \pm 0{,}041$. La Tabla 2 y la Figura 4 traen el resto.

Los barridos se midieron sobre las 736 filas de entrenamiento. En profundidad la accuracy de prueba sube de 0,6127 con tope 1 hasta 0,7677 sin tope, y sigue subiendo en el último punto: no hay un óptimo asentado. La regla igual elige el más chico dentro de un desvío del máximo, que es profundidad 2 ($0{,}7286 \pm 0{,}0344$, 7 nodos). La Figura 6 muestra esa grilla. En gain ratio también sube hasta el umbral 0; el elegido es 0,1 ($0{,}7432 \pm 0{,}0293$). Con umbral 0,2 el árbol queda en un nodo y la accuracy baja a 0,5530. En cantidad de árboles sube

Tabla 2. DT y RF: media y desvío en las 25 particiones.

métrica	DT media	DT desvío	RF media	RF desvío
accuracy	0.7850	0.0265	0.8187	0.0305
precision macro	0.7848	0.0275	0.8185	0.0317
precision weighted	0.7856	0.0269	0.8192	0.0307
recall macro	0.7796	0.0270	0.8145	0.0303
recall weighted	0.7850	0.0265	0.8187	0.0305
F1 macro	0.7809	0.0271	0.8157	0.0308
F1 weighted	0.7840	0.0266	0.8182	0.0304
ROC-AUC	0.8195	0.0295	0.8839	0.0254
tiempo de ajuste (s)	50.5863	9.7970	0.4842	0.0406
tiempo de predicción (s)	0.1510	0.0324	0.1355	0.0228
nodos	452.0000	36.4406	28876.4800	664.6569
hojas	247.3600	18.6299	14488.2400	332.3284
altura	13.0000	0.0000	23.0400	1.3064

Tabla 5. DT con poda pesimista y sin poda: media y desvío en 25 particiones.

métrica	sin poda media	sin poda desvío	con poda media	con poda desvío
nodos	452.0000	36.4406	71.2000	15.7374
hojas	247.3600	18.6299	39.0400	8.4335
altura	13.0000	0.0000	8.8400	0.8981
accuracy train	0.9285	0.0086	0.8663	0.0151
accuracy test	0.7850	0.0265	0.7741	0.0284
brecha	0.1435	0.0310	0.0922	0.0353

hasta 200 (0,8134 ± 0,0322); el elegido es 10 (0,7859 ± 0,0347). La hoja mínima y los atributos por corte sí se frenan: el máximo de la hoja está en 5 ejemplos (0,7772) y la regla elige 50; el máximo de atributos por corte es `sqrt` y la regla elige `log2`.

Sin poda la accuracy de entrenamiento es 0,9285, no 1. La brecha es 0,1435. Con poda pesimista el árbol baja a 71,2 nodos, 39,0 hojas y altura 8,84. La accuracy de entrenamiento queda en 0,8663 y la de prueba en 0,7741. La brecha queda en 0,0922. El bosque memoriza el entrenamiento (accuracy 1) y su brecha es 0,1813, más ancha que la del árbol, con una accuracy de prueba más alta. La Tabla 5 y la Figura 7 usan las mismas 25 particiones. En un solo corte de la misma semilla, la poda pasó de 448 a 71 nodos y la accuracy de prueba de 0,7880 a 0,7989. La brecha de ese corte pasó de 0,1250 a 0,0557. El 90,2 % de las predicciones de prueba coincidió. Con hoja mínima de 10, ese mismo corte tuvo accuracy de prueba 0,8370.

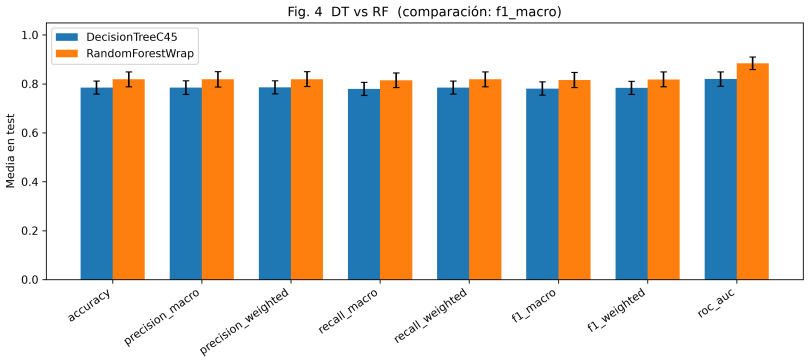


Figura 4. Métricas de prueba, media y desvío, en las 25 particiones.

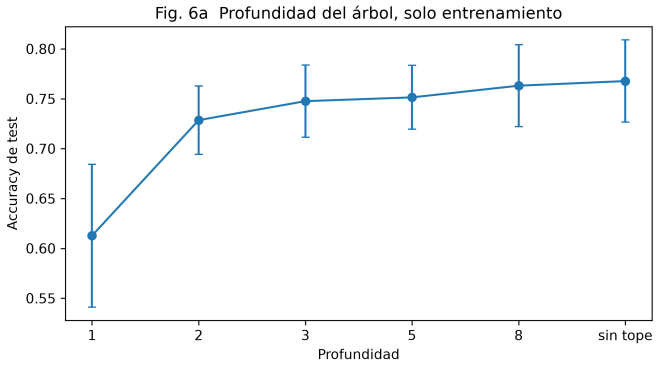


Figura 6. Accuracy de prueba según la profundidad, solo sobre el entrenamiento.

La curva de aprendizaje usa fracciones del entrenamiento. Con 147 filas el árbol tiene accuracy de prueba 0,707 y el bosque 0,771. El bosque llega a 0,823 con 441 filas y cierra en 0,813 con las 736. El árbol cierra en 0,768. La Figura 8 es esa curva. En el árbol ajustado sobre las 736 filas, el atributo que más importa al bosque es `cp` (0,150), seguido de `chol` (0,132) y `exang` (0,116). El árbol usa `exang` en 35 nodos y `cp` en uno. Juntos, y sin haberlos buscado en una grilla cruzada, los valores elegidos dan en el holdout un árbol con accuracy 0,7935, F1 macro 0,7926 y ROC-AUC 0,8057, y un bosque de 10 árboles con `log2` de 0,8370, 0,8336 y 0,8923.

En el holdout de los valores por defecto, con la positiva presencia, el árbol tiene accuracy 0,7880, F1 macro 0,7819 y ROC-AUC 0,8192. El bosque tiene 0,8641, 0,8616 y 0,9219, por encima del bosque chico del párrafo anterior. En las 184 filas de prueba hay 82 ausencia y 102 presencia. La matriz del árbol, con positiva presencia, es TN 57, FP 25, FN 14 y TP 88: se equivoca en 25 de 82 ausencia (30,5 %) y en 14 de 102 presencia (13,7 %). La del bosque es 67, 15, 10

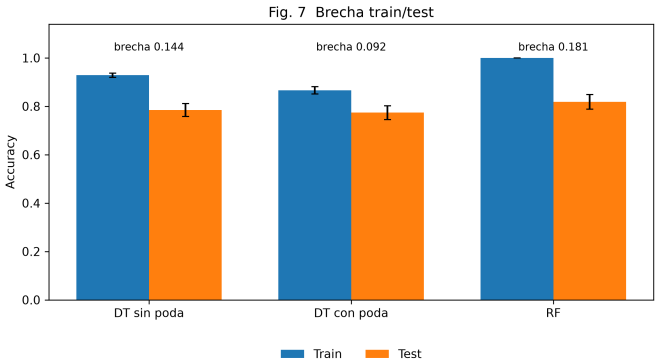


Figura 7. Accuracy de entrenamiento y de prueba. La brecha es la diferencia de las medias.

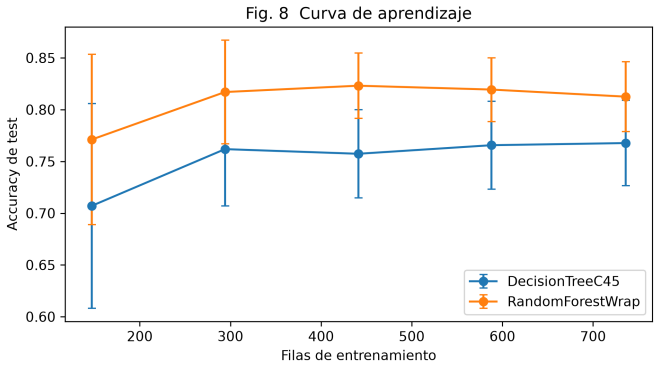


Figura 8. Accuracy de prueba según las filas de entrenamiento.

y 92: 15 de 82 ausencia (18,3 %) y 10 de 102 presencia (9,8 %). Las Figuras 9 y 10 salen de este corte. El mejor corte continuo de un solo atributo, en un test del 30 %, es `chol` $\leq$ 42,50, con gain ratio 0,122 y accuracy 0,5978. El umbral cae entre el 0 y el resto de los colesteroles.

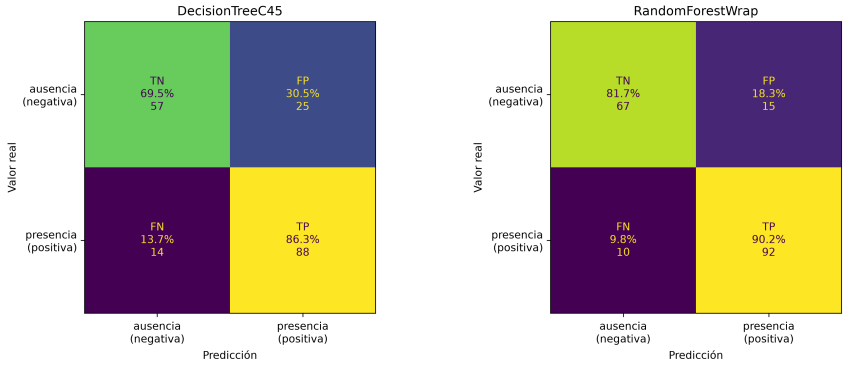


Figura 9. Matrices de confusión del holdout, normalizadas por fila.

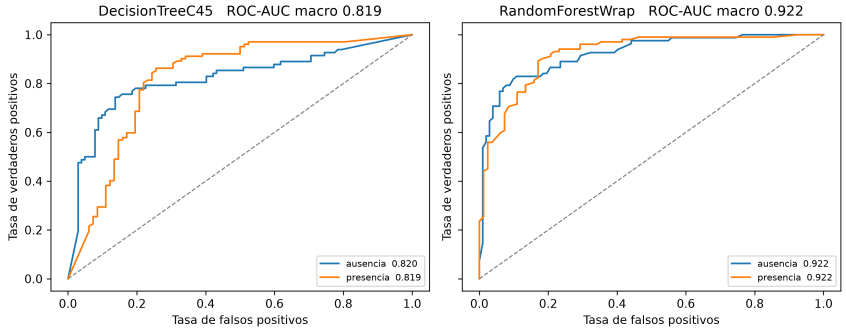


Figura 10. Curva ROC por clase en el holdout. El título de cada panel es el ROC-AUC macro.

6. Discusión

El bosque gana el F1 macro, la accuracy y el ROC-AUC, y se ajusta en una fracción del tiempo. El árbol queda dos órdenes de magnitud más chico. La brecha del bosque es más ancha porque la accuracy de entrenamiento es 1 y la de prueba, aunque mejor, no cierra esa diferencia. La del árbol es más chica porque el entrenamiento tampoco llega a 1: con un solo corte por atributo y con faltantes repartidos, las hojas no quedan todas puras.

La poda se juzga por la media de las 25 particiones. Ahí achica el árbol y la brecha, y la accuracy de prueba baja cerca de un punto. En el holdout de la misma semilla la accuracy de prueba sube de 0,7880 a 0,7989. Las dos lecturas conviven: la decisión de podar mira el entrenamiento, y un corte solo no repite la media.

Tres grillas siguen subiendo en el extremo más complejo. Publicar profundidad 2, gain ratio 0,1 o 10 árboles como «el» óptimo sería más de lo que la regla dice. La regla pide el más chico dentro de la banda, y el informe deja constancia

de que la banda llega hasta el final. Además esos cinco valores se eligieron de a uno. El holdout de la combinación no mejora al bosque por defecto.

`ca` y `thal` faltan en más de la mitad de las filas, sobre todo fuera de Cleveland. El árbol reparte esa masa. El bosque imputa la moda, que sale de las filas donde el valor sí está. Los dos modelos reciben las mismas columnas y no resuelven el faltante del mismo modo. El colesterol 0, dejado a propósito, alcanza para ser el mejor corte de un solo atributo. Ese corte no se lee como un umbral clínico.

El tiempo de ajuste no se lee como una propiedad del algoritmo en abstracto. El árbol está escrito en Python. El bosque llama a una implementación compilada. Los 50,6 segundos frente a 0,48 segundos son el costo medido en esta corrida, con 4 procesos.

7. Conclusiones

En Heart Disease, con las cuatro sedes, clase binaria, semilla 42 y los valores por defecto, el bosque de scikit-learn supera al árbol del equipo en F1 macro, en accuracy y en ROC-AUC, y se ajusta mucho más rápido. El árbol queda en 452 nodos de altura 13. El bosque, en unos 28 900.

La poda pesimista con $CF = 0{,}25$ reduce nodos, hojas, altura y la brecha. En las 25 particiones la accuracy media de prueba baja. En el holdout de la semilla 42, sube.

El árbol implementa gain ratio, umbrales continuos, faltantes fraccionarios y la estimación AddErrs. Sigue en deuda con un C4.5 completo en tres puntos: un solo corte por atributo, poda sin *subtree raising* y un factor t propio. Los barridos se corrieron sobre el entrenamiento. Donde la accuracy sigue subiendo hasta el modelo más grande, el valor elegido por la regla del más chico dentro de un desvío no queda como un óptimo cerrado.

Referencias

1. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)
2. Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J., Sandhu, S., Guppy, K., Lee, S., Froelicher, V.: International application of a new probability algorithm for the diagnosis of coronary artery disease. American Journal of Cardiology **64**, 304–310 (1989)
3. Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R.: Heart disease. UCI Machine Learning Repository (1988), donante: David W. Aha
4. Liu, B.: Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data. Springer, 2 edn. (2011)
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
6. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann, San Mateo, CA (1993)